

Hide-and-Seek: A Multi-Agent Reinforcement Learning Simulator

Jakub Szafranski¹[0009-0006-0548-3461]

Wrocław University of Science and Technology, Faculty of Information and
Communication Technology,
Wrocław, Poland
266498@student.pwr.edu.pl

Abstract. This paper presents a hide-and-seek simulator developed to investigate learning processes in multi-agent systems using tabular reinforcement learning algorithms. The simulator offers a configurable 2D environment with real-time visualization, data logging, and adjustable parameters, including state representations, reward schemes, game rules, and the selection of learning algorithms along with their parameters. It facilitates the analysis of how these factors influence the learning capabilities of the agent. The research examines their impact by comparing performance metrics, such as success rate and average cumulative reward, across configurations. Experimental results show that Monte Carlo underperformed relative to Q-learning and Sarsa. Relative state representations boost both success rates and rewards over full representations. Continuous rewards consistently achieve higher success rates, while terminal ones, though less reliable, can yield higher average returns. The simulator serves as a versatile tool for both educational and research purposes into multi-agent reinforcement learning.

Keywords: Reinforcement Learning · Multi-Agent Systems · Hide-and-Seek

1 Introduction

Reinforcement learning (RL) involves agents learning to map situations to actions to maximize a numerical reward through trial-and-error exploration, where actions influence immediate and future outcomes [1]. In multi-agent systems [2], several agents coexist and interact within a shared environment, meaning an agent's behavior is shaped not only by the environment but also by the actions of others. This dynamic is exemplified in a hide-and-seek game, where a seeker and a hider adapt their strategies based on mutual interactions. This work introduces a configurable simulator designed to study such learning processes in a multi-agent hide-and-seek scenario, analyzing their impact through evaluation of success rates and cumulative rewards. Reinforcement learning has been widely applied to various game-based environments, proving its effectiveness as a framework for developing intelligent game-playing agents. This approach spans

simple games, such as Snake [3] or Pac-Man [4], to complex environments such as Minecraft [5] or Gran Turismo [6]. Multi-agent hide-and-seek using RL has been explored by researchers investigating how agents optimize seeking and hiding behaviors through tabular [7] and policy gradient [8] methods, demonstrating effectiveness in competitive multi-agent environments.

2 Simulation Environment

The hide-and-seek simulator operates on a static, 2D $N \times M$ grid populated with obstacles, as shown in Figure 1. The seeker attempts to locate the hider within a limited field of view, while the hider aims to avoid detection. The game ends when the seeker finds the hider or after a configurable maximum number of moves is reached, at which point the hider is declared the winner if undetected. In each turn, both agents select one of the five available actions: moving up, down, left, right, or staying in place.

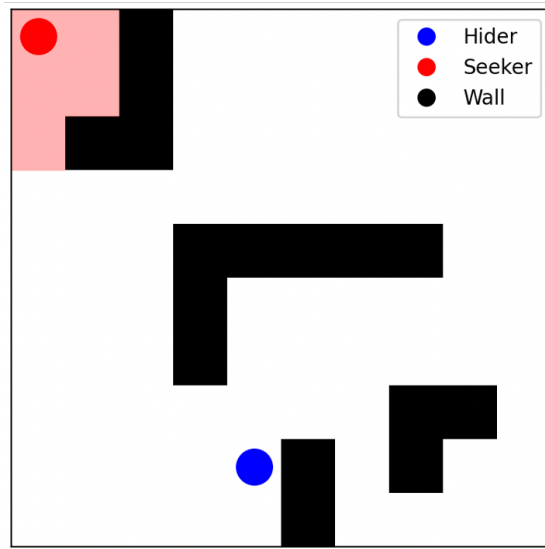


Fig. 1. Visualization of the hide-and-seek environment.

The simulator, implemented in Python, offers a versatile platform supporting:

- Configuration of rules (e.g., seeker’s range of view, maximum move limit) and grid layout.
- Various state definitions:
 - Relative position: The agent is aware of its own coordinates (x, y) and the relative direction of its opponent (e.g., left, right, below, above).

- Full positional knowledge: The agent has access to its own coordinates (x, y) and the exact coordinates (x, y) of its opponent.
- Reward schemes:
 - Continuous: Seeker receives -1 per move, hider receives +1 per move, winner receives +30, and loser receives -30.
 - Terminal: Winner receives +30, and loser receives -30 at the end of the game.
- Reinforcement learning algorithms: Q-learning, Sarsa, and Monte Carlo Control [1].
- Action selection strategies: ϵ -greedy [1] with fixed or linearly decaying ϵ .
- Data logging: Each episode’s data—length, agents’ actions and returns, and the winner—is recorded in a JSON file.
- Real-time visualization.

The hide-and-seek simulator is open-source and accessible at the GitHub repository <https://github.com/jakub-szafranski/marl-hide-and-seek>.

3 Research Scope

This study explores the effects of simulator parameters on agent learning in the hide-and-seek environment, measuring learning effectiveness through success rate and average cumulative reward, reflecting agent performance and policy quality. The investigation focuses on:

- Comparing algorithms: Q-learning, Sarsa, and Monte Carlo Control.
- Assessing state definitions: relative position, and full positional knowledge.
- Evaluating reward schemes: continuous and terminal.

Each configuration undergoes a training phase of 5,000 episodes, based on observed stabilization in preliminary runs. Performance is then evaluated over 1,000 episodes with fixed policies, and the results are averaged to mitigate randomness. This process is repeated 50 times for each configuration to ensure statistical significance of the evaluation. The initial positions of the seeker and hider are randomized, while other parameters remain constant to isolate the effects of the investigated factors. In all configurations, only the seeker agent is learning, while the hider uses a fixed, pre-trained policy. Statistical tests were conducted using the Mann-Whitney U test at a significance level of 0.05. To account for multiple comparisons, a Bonferroni correction was applied.

4 Results

The experiments revealed distinct patterns in agent learning behaviors across different configurations. Table 1 summarizes the key performance metrics for seeker agent under various learning conditions.

Table 1. Performance metrics for seeker agent across configurations (mean $\pm$ SD).

Algorithm	State	Reward	Success Rate	Avg. Reward
Q-learning	Full	Terminal	69.27% $\pm$ 12.25%	19.27 $\pm$ 11.76
	Full	Continuous	76.79% $\pm$ 11.97%	10.94 $\pm$ 12.59
	Relative	Terminal	86.12% $\pm$ 7.54%	36.12 $\pm$ 7.28
	Relative	Continuous	89.97% $\pm$ 4.81%	25.51 $\pm$ 5.48
Sarsa	Full	Terminal	72.23% $\pm$ 10.46%	22.23 $\pm$ 9.67
	Full	Continuous	80.09% $\pm$ 11.65%	14.45 $\pm$ 12.25
	Relative	Terminal	87.98% $\pm$ 6.80%	37.98 $\pm$ 5.98
	Relative	Continuous	91.93% $\pm$ 5.90%	27.83 $\pm$ 6.52
Monte Carlo	Full	Terminal	29.80% $\pm$ 13.52%	-20.20 $\pm$ 11.84
	Full	Continuous	34.43% $\pm$ 14.80%	-34.36 $\pm$ 15.57
	Relative	Terminal	19.20% $\pm$ 13.14%	-30.80 $\pm$ 12.36
	Relative	Continuous	16.84% $\pm$ 13.88%	-52.64 $\pm$ 15.18

A primary observation is that the Monte Carlo method consistently underperformed across all configurations, exhibiting the lowest outcomes in both success rate and average reward. Statistical analysis was conducted using the Mann-Whitney U test at a significance level of 0.05, confirming that Monte Carlo’s success rates and average rewards were significantly lower than those of Q-learning and Sarsa across all configurations, highlighting its difficulty in achieving effective policy convergence.

The comparison of reward schemes reveals that continuous rewards consistently yield higher success rates than terminal rewards, suggesting that incremental feedback facilitates the agent’s ability to reach its objective. Mann-Whitney U test at a significance level of 0.05 confirmed that continuous rewards produced significantly higher success rates than terminal rewards across all algorithms and state representations. However, an intriguing pattern emerges: terminal rewards, despite lower success rates, produce higher average rewards. The Mann-Whitney U test further validated that terminal rewards resulted in significantly higher average rewards than continuous rewards, indicating that while the agent is less likely to succeed with terminal rewards, successful episodes are completed more efficiently.

Another notable finding is the superior performance of relative state representations over full state representations across most configurations. Statistical analysis using the Mann-Whitney U test at a significance level of 0.05 confirmed that relative state representations yielded significantly higher success rates and average rewards than full state representations for Q-learning and Sarsa. This may be attributed to the relative state representation requiring a Q-table four times smaller, which facilitates faster convergence. However, while relative states accelerate learning in the short term, full state representations could potentially outperform given sufficient training steps, as they encapsulate more comprehensive environmental information.

Overall, Sarsa and Q-learning combined with relative state representations and continuous rewards achieved the highest success rates, while the combination of relative state representations and terminal rewards yielded the highest average rewards.

5 Conclusion

The hide-and-seek simulator was implemented to explore multi-agent reinforcement learning dynamics. The simulator provides a versatile platform for studying agent learning, combining high configurability with real-time visualization. It can be effectively used in educational settings to teach concepts of reinforcement learning, multi-agent systems, and algorithm optimization through hands-on experimentation. Conducted experiments validate its effectiveness, demonstrating the influence of algorithm selection and reward structure on learning performance. Future enhancements could extend the platform's capabilities by incorporating deep reinforcement learning algorithms and introducing multiple seekers and hiders as team members, facilitating a fully dynamic environment and enabling the study of agent cooperation.

Disclosure of Interests. The author has no competing interests to declare that are relevant to the content of this article.

References

1. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction. 2nd edn. The MIT Press, Cambridge, Massachusetts (2018).
2. G. Weiss, Ed., Multiagent Systems: A Modern Approach to Distributed Artificial Intelligence. Cambridge, MA: MIT Press, 1999.
3. Ray, D., Ghosh, A., Ojha, M., Singh, K.P.: Deep Q-Snake: An Intelligent Agent Mastering the Snake Game with Deep Reinforcement Learning. In: TENCON 2024 - 2024 IEEE Region 10 Conference (TENCON), pp. 1464–1469. IEEE, Singapore (2024).
4. An, J., Du, Y.: Training Agent to Play Pac-Man under Authentic Environment Based on Image Recognition. In: 2022 5th International Conference on Pattern Recognition and Artificial Intelligence (PRAI), pp. 68–75. IEEE, Chengdu (2022).
5. Kobayashi, K., Kishi, F.: A Minecraft Agent Based on Hierarchical Deep Reinforcement Learning Model. In: 2024 SICE Festival with Annual Conference (SICE FES), pp. 570–575. SICE, Kochi City (2024).
6. Fuchs, F., Song, Y., Kaufmann, E., Scaramuzza, D., Dür, P.: Super-Human Performance in Gran Turismo Sport Using Deep Reinforcement Learning. IEEE Robotics and Automation Letters **6**(3), 4257–4264 (2021).
7. Jagtap, S., Thomas, A., Gadiwan, S., Mishra, S.: Multi-Agent Reinforcement Learning-Implementation of Hide and Seek. In: 2022 IEEE 3rd Global Conference for Advancement in Technology (GCAT), pp. 1–7. IEEE, Bangalore (2022).
8. Wafa, H., Santoso, J.: Multiagent Simulation on Hide and Seek Games Using Policy Gradient Trust Region Policy Optimization. In: 2020 7th International Conference on Advance Informatics: Concepts, Theory and Applications (ICAICTA), pp. 1–5. IEEE, Tokoname (2020).